Smart File Organizer - Complete Project Walkthrough

Summer Internship Project Presentation Guide

PART 1: WHAT IS THIS PROJECT & WHY COMPANIES NEED IT

What Does This Tool Do?

Smart File Organizer is a **command-line tool (CLI)** built in Python that automatically:

- Scans any folder on your computer and tells you what's inside
- Sorts messy files into organized folders by type and date
- Finds duplicate files that are wasting storage space
- Reads inside files (CSVs, images, text) to extract useful information
- Generates professional reports (HTML, JSON, CSV) of everything it found

Real-World Problem It Solves (Why Companies Care)

In any IT company, every team generates thousands of files:

- **HR Department** - Resumes, offer letters, ID proofs, attendance sheets
- **Engineering Team** - Source code, logs, build artifacts, screenshots
- **Marketing Team** - Images, videos, presentations, campaign CSVs
- **Finance** - Excel sheets, invoices, tax documents

The 3 Problems:

Problem 1: Messy Folders Everyone dumps files in Downloads, Desktop, or shared drives with no structure. Finding a specific file becomes like searching for a needle in a haystack.

Problem 2: Duplicate Files Wasting Storage People copy the same file multiple times. On cloud storage (AWS S3, Google Drive), this directly costs money. A company with 500 employees might waste thousands of rupees monthly on duplicate storage.

Problem 3: No Visibility Into Data A manager gets a folder with 200 CSV files. They have no idea how many rows each has, what columns exist, or which ones are useful — without opening every single file.

How Our Tool Solves This:

Problem	Our Solution
Messy folders	Auto-organizes into `Category/Year/` structure
Duplicate files	SHA-256 hashing finds exact duplicates, shows recoverable space
No visibility	Extracts metadata from CSVs/images, generates HTML dashboard

Problem	Our Solution
Risk of data loss	Dry-run mode previews changes before executing, Undo feature to rollback

Use Cases in Companies:

1. **IT Admin** runs it on shared drives monthly to clean up storage
2. **DevOps** uses it to organize log files and build artifacts
3. **Data Team** uses it to catalog and summarize CSV datasets
4. **HR** uses it to organize employee documents by category and year

---

PART 2: TECH STACK (What Technologies Are Used)

Languages & Libraries

Technology	What It Is	Why We Used It
Python 3.10+	Programming language	Industry standard for automation & scripting tools
Click	CLI framework library	Creates professional command-line interfaces with help menus, options, flags
Pandas	Data analysis library	Reads CSV files, counts rows/columns, calculates statistics (mean, min, max)
Pillow (PIL)	Image processing library	Extracts image dimensions, format, EXIF data (camera model, date taken)
PyYAML	YAML parser	Loads configuration from `config.yaml` so users can customize without changing code
hashlib	Built-in Python module	Generates SHA-256 hash of file contents to detect exact duplicates
pathlib	Built-in Python module	Handles file paths across Windows/Mac/Linux
Pytest	Testing framework	Runs 71 automated unit tests to ensure code reliability

Key Concepts Used

1. SHA-256 Hashing (for Duplicate Detection)

SHA-256 is a cryptographic algorithm that converts any file's content into a unique 64-character string (called a "hash"). If two files have the exact same content (even if different names), they produce the same hash.

```
notes.txt      -> hash: a1b2c3d4e5f6...
duplicate.txt  -> hash: a1b2c3d4e5f6...  (Same! It's a duplicate)
```

```
report.csv     -> hash: x9y8z7w6v5u4...  (Different content, different hash)
```

2. Dry-Run Pattern (Safety Feature)

Before moving or deleting any file, the tool shows a preview of what it WOULD do. The user confirms by adding `--execute` flag. This is a common pattern in enterprise tools (like Terraform, Ansible) where mistakes can be costly.

3. Manifest-Based Undo

When files are organized, the tool saves a `manifest.json` recording every move:

```
{
  "timestamp": "2026-08-07T14:00:00",
  "mode": "copy",
  "operations": [
    {"source": "Downloads/photo.jpg", "destination":
"Organized/Images/2026/photo.jpg"}
  ]
}
```

The `undo` command reads this file and reverses every operation.

4. Modular Architecture

Code is split into 7 independent modules. Each module does ONE job:

```
smart_organizer/
    __init__.py       --> Package initialization, version info
    __main__.py       --> Entry point (allows `python -m smart_organizer`)
    cli.py            --> Click CLI interface (handles user commands)
    scanner.py        --> File scanning & classification logic
    organizer.py      --> File organization engine (move/copy/undo)
    deduplicator.py   --> Duplicate detection using SHA-256
    extractor.py      --> Metadata extraction (CSV stats, image EXIF)
    reporter.py       --> Report generation (JSON, CSV, HTML)
    config.py         --> YAML configuration loader
    utils.py          --> Shared helper functions (logging, formatting)
```

PART 3: CODE EXPLANATION (How Each Module Works)

Module 1: scanner.py

Purpose: Walk through a folder and identify every file.

How it works:

- Uses Python's `pathlib` to recursively walk through directories
- For each file found, it checks the extension (`.jpg`, `.py`, `.csv`, etc.)
- Maps the extension to a category using rules from `config.yaml`
- Creates a `FileInfo` object (dataclass) storing name, path, size, category, dates
- Returns a list of all FileInfo objects

Key function: `scan_directory(path, config, recursive=True)`

```
# Example: .jpg -> "Images", .py -> "Code", .csv -> "Data"
categories = {
    "Documents": [".pdf", ".doc", ".txt"],
    "Images":    [".jpg", ".png", ".gif"],
    "Code":      [".py", ".js", ".java"],
    "Data":      [".csv", ".json", ".xml"],
}
```

Module 2: deduplicator.py

Purpose: Find files that are exact copies of each other.

How it works:

- Takes the list of scanned files
- Reads each file in binary mode, chunk by chunk (8KB at a time)
- Feeds each chunk to SHA-256 hash algorithm
- Groups files by their hash value
- Any group with 2+ files = duplicates found

Key function: `hash_file(file_path, algorithm="sha256")`

```
# Reads file in chunks to handle large files without running out of memory
hasher = hashlib.sha256()
with open(file_path, "rb") as f:
    while chunk := f.read(8192):
        hasher.update(chunk)
return hasher.hexdigest()  # Returns "a1b2c3d4..."
```

Module 3: organizer.py

Purpose: Move or copy files into an organized folder structure.

How it works:

- Takes scanned files and a rename pattern (e.g., `{date}_{original}`)

- Builds destination path: `Organized/Category/Year/renamed_file`
- In dry-run mode: just prints what it would do
- In execute mode: actually creates folders and copies/moves files
- Saves a `manifest.json` for undo capability

Key function: `organize_files(files, output_dir, pattern, mode, dry_run)`

Output structure example:

```
Organized/
    Documents/
        2026/
            2026-08-07_notes.txt
            2026-08-07_report.pdf
    Images/
        2026/
            2026-08-07_photo.jpg
    Code/
        2026/
            2026-08-07_script.py
    manifest.json
```

Module 4: extractor.py

Purpose: Read inside files and extract useful metadata.

How it works:

- For **CSV files**: Uses Pandas to read the file, counts rows, columns, column names, and calculates numeric statistics (mean, min, max for each number column)
- For **Images**: Uses Pillow to get dimensions (width x height), format (PNG/JPEG), color mode (RGB/RGBA), and EXIF data (camera model, date taken)
- For **Text files**: Counts lines, words, characters, detects encoding

Key function: `extract_metadata(file_path, category)`

Module 5: reporter.py

Purpose: Generate summary reports in multiple formats.

How it works:

- Collects scan summary, duplicate summary, and extracted metadata
- **JSON report**: Dumps everything into a structured JSON file
- **CSV report**: Creates a flat CSV with key metrics
- **HTML report**: Generates a self-contained HTML page with inline CSS styling, category charts, and duplicate analysis — can be opened directly in a browser

Key function: `generate_report(scan_summary, format="json")`

Module 6: cli.py

Purpose: The user interface — handles all terminal commands.

How it works:

- Uses the Click library to define commands: `scan`, `organize`, `dedupe`, `report`, `undo`
- Each command has options like `--path`, `--dry-run`, `--format`, `--mode`
- Calls the appropriate module functions and displays formatted results
- Handles errors gracefully with colored output

Module 7: config.py

Purpose: Load settings from `config.yaml`.

How it works:

- Reads the YAML file using PyYAML
- Returns a dictionary of settings (categories, excluded dirs, hash algorithm)
- If no config file exists, uses sensible defaults
- Allows users to customize the tool without touching any Python code

---

PART 4: PROJECT STRUCTURE (Full File Layout)

```
SmartFileOrganizer/
|
|-- smart_organizer/           # Main Python package
|   |-- __init__.py            # Package init, version = "1.0.0"
|   |-- __main__.py            # Entry point for `python -m smart_organizer`
|   |-- cli.py                 # CLI commands (scan, organize, dedupe, report,
undo)
|   |-- scanner.py             # File scanning & classification
|   |-- organizer.py           # File organization with undo support
|   |-- deduplicator.py        # SHA-256 duplicate detection
|   |-- extractor.py           # Metadata extraction (CSV, Image, Text)
|   |-- reporter.py            # Report generation (JSON, CSV, HTML)
|   |-- config.py              # YAML config loader
|   |-- utils.py               # Logging, formatting helpers
|
|-- tests/                     # Unit test suite (71 tests)
|   |-- __init__.py
|   |-- test_scanner.py        # 16 tests for scanner module
|   |-- test_deduplicator.py   # 13 tests for deduplicator
|   |-- test_organizer.py      # 10 tests for organizer
|   |-- test_extractor.py      # 18 tests for extractor
|   |-- test_reporter.py       # 14 tests for reporter
|
|-- sample_data/               # Demo files for testing
|   |-- report.csv             # Sample CSV with employee data
```

```
|   |-- notes.txt              # Sample text file
|   |-- duplicate.txt          # Exact copy of notes.txt (for dedup demo)
|   |-- code_sample.py         # Sample Python script
|   |-- data.json              # Sample JSON data
|
|-- config.yaml                # Configuration file
|-- requirements.txt           # Python dependencies
|-- setup.py                   # Package installation setup
|-- README.md                  # Project documentation
```

PART 5: HOW TO USE IT (Step-by-Step Commands)

Prerequisites (One-Time Setup)

Step 1: Open Terminal

Open **PowerShell** or **Command Prompt** on your laptop.

Step 2: Go to Project Folder

```
cd C:\Users\jaypa\Documents\SmartFileOrganizer
```

Step 3: Install Dependencies (if not already installed)

```
pip install -r requirements.txt
```

This installs: click, pandas, Pillow, PyYAML, pytest

Command 1: See All Available Commands

```
python -m smart_organizer --help
```

What it shows: List of all commands (scan, organize, dedupe, report, undo) with descriptions.

Command 2: Scan a Directory

```
python -m smart_organizer scan --path ./sample_data
```

What it does: Scans the `sample_data` folder and shows:

- Total number of files and their combined size
- Category breakdown (how many Documents, Code, Data files)
- Extension breakdown (how many .txt, .py, .csv, .json files)

Expected Output:

```
[FILES] Total Files:  5
[SIZE]  Total Size:   3.48 KB

CATEGORY BREAKDOWN
--------------------------------------------------
Category          | Count    | Size
[DAT] Data        | 2        | 842 B
[DOC] Documents   | 2        | 2.16 KB
[CODE] Code       | 1        | 507 B
```

Command 3: Detect Duplicate Files

```
python -m smart_organizer dedupe --path ./sample_data
```

What it does: Computes SHA-256 hash of every file, groups identical ones, shows:

- How many duplicate groups found
- How much space you could recover by deleting duplicates
- Which file to KEEP and which is the DUPE

Expected Output:

```
[SEARCH] Duplicate Groups:   1
[FILES]  Duplicate Files:    1
[SIZE]   Space Recoverable:  1.08 KB

Group 1 | Size: 1.08 KB | Copies: 2
  [KEEP]   notes.txt
  [DUPE]   duplicate.txt
```

Command 4: Organize Files (Dry Run - SAFE PREVIEW)

```
python -m smart_organizer organize --path ./sample_data
```

What it does: Shows what the tool WOULD do without actually moving anything. This is the default safe mode.

Expected Output:

```
[DRY RUN] No files will be moved

[COPY] code_sample.py
     -> Organized/Code/2026/2026-08-07_code_sample.py
[COPY] report.csv
     -> Organized/Data/2026/2026-08-07_report.csv
```

Command 5: Organize Files (Actually Execute)

```
python -m smart_organizer organize --path ./sample_data --execute --mode copy
```

What it does: Actually copies files into the `Organized/` folder structure.

- `--execute` = do it for real (not just preview)
- `--mode copy` = copy files (originals stay in place)
- `--mode move` = move files (originals are removed)

After running, check the new folder:

```
dir Organized
```

Command 6: Undo the Organization

```
python -m smart_organizer undo
```

What it does: Reads the `manifest.json` and reverses every file operation. Files go back to where they were before organizing.

Command 7: Generate HTML Report

```
python -m smart_organizer report --path ./sample_data --format html
```

What it does: Creates a beautiful HTML report file. Open it in Chrome/Edge to see a styled dashboard with:

- File summary statistics
- Category breakdown

- Duplicate analysis
- Extracted metadata from CSVs and images

Other report formats:

```
python -m smart_organizer report --path ./sample_data --format json
python -m smart_organizer report --path ./sample_data --format csv
```

Command 8: Run Unit Tests (Show Code Quality)

```
python -m pytest tests/ -v
```

What it does: Runs all 71 automated tests. Shows each test passing with green checkmarks. This proves the code is tested and reliable.

Command 9: Scan Any Folder on Your Laptop

You can point the tool at any folder on your system:

```
python -m smart_organizer scan --path C:\Users\jaypa\Downloads
python -m smart_organizer scan --path C:\Users\jaypa\Desktop
```

PART 6: QUICK CHEAT SHEET FOR PRESENTATION

What to Show	Command	Time
Help menu	`python -m smart_organizer --help`	30 sec
Scan sample data	`python -m smart_organizer scan --path ./sample_data`	30 sec
Find duplicates	`python -m smart_organizer dedupe --path ./sample_data`	30 sec
Preview organize	`python -m smart_organizer organize --path ./sample_data`	30 sec
Execute organize	`python -m smart_organizer organize --path ./sample_data --execute --mode copy`	30 sec

What to Show	Command	Time
Undo organize	`python -m smart_organizer undo`	15 sec
HTML report	`python -m smart_organizer report --path ./sample_data --format html`	30 sec
Run all tests	`python -m pytest tests/ -v`	15 sec

Total demo time: ~4 minutes

PART 7: COMMON QUESTIONS & ANSWERS

Q: Why Python and not Java/JavaScript? A: Python is the industry standard for automation, scripting, and data processing tools. Libraries like Pandas and Pillow make data extraction very easy.

Q: Why CLI and not a GUI? A: CLI tools are preferred in enterprise environments because they can be automated, scheduled via cron jobs, and integrated into CI/CD pipelines. They also work on servers without a display.

Q: What makes this different from just manually sorting files? A: Scale and consistency. This tool can process thousands of files in seconds with consistent naming conventions, something that would take hours manually and be error-prone.

Q: How does it handle large files? A: The hashing reads files in 8KB chunks instead of loading the entire file into memory. This means it can handle files of any size without crashing.

Q: Can this be used on Linux/Mac? A: Yes. We used `pathlib` instead of hardcoded Windows paths, making it cross-platform.